```
Output:
int(4)
int(2)
```

As you might expect, `strpos()` returns 4, because `phe ve` is indeed four bytes from the start of `$quote`. `mb_strpos()` returns 2 because it is aware of UTF-8 character encoding and is able to evaluate the first four bytes as two distinct characters. The `mb_str*pos()` functions will accept an optional offset value as a third argument and are also able to work with other multi-byte encodings, which can be specified as an optional fourth argument, in case you need to work with something other than UTF-8.

strspn(), strcspn()

If you are interested in finding how many bytes at the start of a string match a set of values (or do not match, as the case may be), but are not concerned about an exact sub-string match, it is possible to define a collection of bytes, known as a mask, which the sub-string must contain, and use the `strspn()` and `strcspn()` functions.

```
$book = 'ISBN-978-1484267905 PHP 8 Objects, Patterns, and Practice';
var_dump(strspn($book, '.-BINSO0123456789'));
```

```
Output:
int(19)
```

Here we define a mask that contains all of the (ASCII) characters in the sub-string `ISBN-978-1484267905`, and `strspn()` returns `19`, which means the length of the sub-string containing only the bytes defined in the mask is nineteen.

To achieve the same outcome but with inverted logic, use `strcspn()`:

```
var_dump(strcspn($book, ' '));
```

```
Output:
int(19)
```

Here we've determined that the first twenty bytes do not contain an ASCII space character. Note that both `strspn()` and `strcspn()` are binary safe…

```
$binaryStr = "ab\x00cdef";
var_dump(strspn($binaryStr, "a\x00bc"));
var_dump(strcspn($binaryStr, "def"));
```